

The Unlabeled Crowd

*A novel of distances, k-means, medoids, dendrograms,
silhouettes and nearest neighbours*

Book Two of The Model That Lied

A story retelling of the lecture

Fundamentals of Machine Learning: Unsupervised Learning

by R. Billot, G. Bella & P. Lenca, IMT Atlantique, 2026

Contents

- 1. No Labels 4
- 2. How Far Is Near? 8
- 3. Counting Partitions 13
- 4. Moving Centres 16
- 5. The Medoid 22
- 6. The Family Tree 26
- 7. How Many? 31
- 8. Salmon and Sea Bass 36
- Epilogue: Six Groups 42
- Appendix: Cheat Sheet 43
- Appendix: Map to the Lecture 45

Each chapter ends with a box, What Gaël learned, that sums up the ideas. Coral boxes give algorithms in pseudo-code; violet boxes give short Python examples you can run with NumPy, SciPy and scikit-learn. Every number in the worked examples was computed, not guessed. At the back: a cheat sheet, a map to the lecture slides, and further reading.

Characters

Léa, data scientist at the Banque de l'Iroise, Brest; heroine of Book One

Gaël, her intern, twenty-two, keen, and new to unsupervised learning

Madame Kerguelen, officially retired, unofficially in the office three days a week

Jean-Yves Le Bihan, the bank's director

Anna, Bruno and Chloé, three customers who teach everyone about units

The billionaire, who, for reasons of his own, has a student account

*Previously, in *The Model That Lied*: Léa learned, the hard way, that a fraud detector with 99% accuracy can catch no fraud at all, and how to evaluate a supervised classifier honestly — bias and variance, test sets and cross-validation, precision and recall, ROC curves. This book begins a year later, with a problem that has no labels at all.*

No Labels

A year after the fraud system went live, Léa had a desk by the window, the wobbly chair, and an intern of her own. Gaël was twenty-two, quick, and still carried his laptop open in front of him like a waiter carrying a cake. She had decided this was a good sign. It meant he was excited about something.

“The director wants a segmentation of our customers,” he announced one Monday in October. “He says marketing wants to send different offers to different kinds of people. I’ve already started. What’s the label?”

“There isn’t one.”

Gaël waited for the rest of the sentence. It didn’t come.

“But last year’s fraud data had labels,” he said. “Fraud, not fraud. You trained a model to predict them. What do I predict this time?”

“Nothing. Nobody has ever sat down and written next to each of our two hundred thousand customers ‘student’, ‘young family’, ‘retired traveller’. Those groups might exist. They might not. They might be completely different groups that nobody in marketing has thought of. Finding out is the job.”

Across the room, Madame Kerguelen — officially retired since June, unofficially in the office three days a week “to keep an eye on things” — lowered her newspaper.

“Welcome,” she said to Gaël, “to **unsupervised learning**.”

. . .

They took him to Océanopolis, the great aquarium on the edge of the harbour, because Kerguelen said you couldn't explain classification without animals. They stopped in front of a wall where a century of naturalists had arranged every creature of the Breton coast into a tree: kingdom, phylum, class, order, family, genus, species.

"Somebody built this by hand," Kerguelen said. "Looking at thousands of animals, noticing which ones resemble each other, grouping them, then grouping the groups. Nobody told Linnaeus the answer in advance. He had no labels. He had only resemblance."

"In English they call what we're about to do **clustering**," Léa told Gaël. "In France you'll hear *classification automatique* or *classification non supervisée*. Be careful with that word. In English machine learning, 'classification' almost always means the *supervised* problem, predicting a known label like fraud. Same word, different job. And don't confuse either of them with *ranking*, which is about putting things in order."

"Every field has its own name for it, too," Kerguelen added. "Doctors classifying diseases call it *nosology*. Biologists call it *taxonomy*. Marketing people, like the ones upstairs, call their result a *typology*. It's all the same activity: take a crowd of individuals and organize it into groups that make sense."

On the way out they passed a poster of the night sky, and Kerguelen stopped again. "Astronomers did it with stars. They sorted thousands of stellar spectra into types by resemblance — O, B, A, F, G, K, M — before anyone understood the physics behind them. Later it turned out the types lined up with temperature. The classification came first. The understanding followed." She tapped the glass. "That's the promise of clustering. Not predicting a label you already have. *Revealing a structure you didn't know was there.*"

. . .

Back at the office, Kerguelen pulled up a chair next to Gaël. "Before any algorithm, there's the table. Show me what you've got."

He showed her: one row per customer, one column per variable. Age, income, number of card payments per month, average basket, share of spending abroad, number of products held, and so on.

“Good. That’s the starting point of almost every method you’ll see. A numerical table with N rows and P columns. The rows are the **individuals** — also called observations, or examples — and together they form a set we’ll call Ω . The columns are the **variables**, or attributes. You have N equal to two hundred thousand and P equal to about thirty.”

$$\begin{aligned} \text{data} &= \text{table of size } N \times P \\ \Omega &= \{x_1, \dots, x_N\}, \text{ each } x_i \in \mathbb{R}^P \end{aligned}$$

“And the goal?”

“The goal is to **organize Ω into homogeneous, natural classes**, without any hypothesis about what those classes should be. Homogeneous means that individuals inside a class resemble each other. Natural means the classes are really in the data, not imposed by us. If it works, it reveals the structure of the data and helps us understand it. Clustering belongs to a larger family of methods called *data analysis*: ways of reducing a big table to a simpler picture of it.”

. . .

Kerguelen opened a famous old dataset on her own laptop: the irises that the statistician Ronald Fisher published in 1936. A hundred and fifty flowers, four measurements each. She plotted just two of them, sepal width against petal length, all in the same grey.

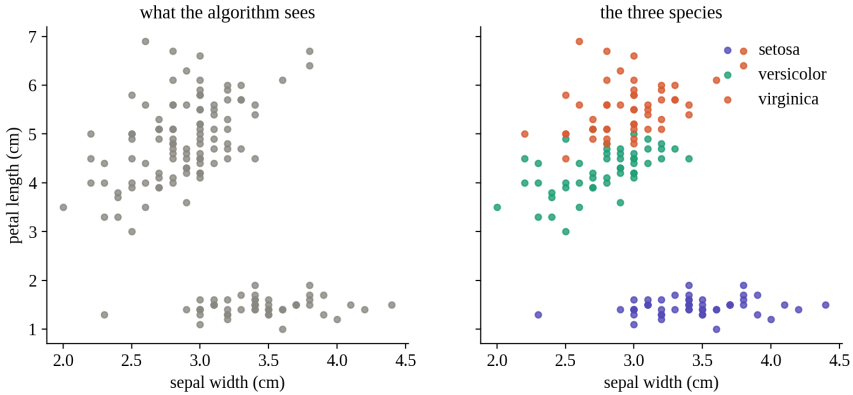
“How many natural classes do you see?”

Gaël leaned in. “Two. There’s a little clump at the bottom, and a big cloud on top.”

“Léa?”

“I’d say two as well. Maybe the big cloud is really two clouds pressed together.”

“There are three species in this dataset,” Kerguelen said. “Setosa, versicolor and virginica. The biologists are quite sure.” She coloured the points by species. The little clump was setosa, all alone. The big cloud was versicolor and virginica, overlapping so much that nobody could have drawn a clean line between them by eye.



Fisher's irises, sepal width against petal length. Left: what the algorithm sees, with no labels. Right: the three species. Setosa stands alone; versicolor and virginica overlap.

“So the answer is three,” said Gaël.

“The answer to *which species are these* is three. The answer to *how many groups does the data itself show* is less clear. Maybe two. Maybe three, if you use all four measurements. Maybe more, if some subgroup hides inside a species.” She closed the laptop. “Remember this picture. We’ll come back to it at the very end, and you’ll find that even the cleverest mathematics can’t settle it for you. There’s no teacher here holding the right answer. That’s what *unsupervised* means.”

“Then how do we even start?”

“With the most basic question of all,” Kerguelen said. “Grouping similar things together means you need to know when two things are similar. So: how far apart are two customers?”

What Gaël learned

Unsupervised learning has no labels. Clustering (in French, *classification automatique* or *non supervisée*) organizes a set of individuals into homogeneous, natural classes with no prior definition of the classes; it reveals structure. It goes by other names in other fields: nosology, typology, taxonomy. The starting point is an $N \times P$ table of individuals and variables. How many natural classes exist is often a genuinely open question.

How Far Is Near?

Kerguelen wrote three customers on the whiteboard.

<i>customer</i>	<i>age</i>	<i>income (€ / year)</i>
Anna	25	30,000
Bruno	60	31,000
Chloé	26	45,000

Three customers, two variables with very different units.

“Which two are most alike?”

“Anna and Chloé,” Gaël said at once. “Both about twenty-five. Anna and Bruno are thirty-five years apart.”

“Now compute it. The ordinary distance. The one you learned at school.”

Gaël typed. “Euclidean distance: square the difference on each variable, add them up, take the square root. Anna to Bruno: thirty-five squared plus a thousand squared... about one thousand. Anna to Chloé: one squared plus fifteen thousand squared...” He stopped. “Fifteen thousand. The computer says Anna is fifteen times closer to a sixty-year-old than to a girl her own age.”

“Because income is measured in euros and age in years. A difference of a thousand euros is a tiny thing, and a difference of thirty-five years is enormous, but to the formula a thousand is simply bigger than thirty-five. Whichever variable has the largest numbers takes over the whole distance.”

$$\text{Euclidean distance: } d(x, y) = \sqrt{(\sum_{j=1..P} (x_j - y_j)^2)}$$

“So we rescale.”

“We usually **standardize**: for each variable, subtract its mean and divide by its standard deviation. Every column then speaks the same language, ‘how many standard deviations from average’. Do it.”

Gaël did. Anna to Bruno: 2.16. Anna to Chloé: 2.19.

“Now they’re about equally far,” Léa said. “Anna shares an income with Bruno and an age with Chloé, and both differences count the same.”

“Which is not the same as *correct*,” Kerguelen said. “Maybe marketing thinks age matters twice as much as income. Then you’d weight it. The point is that *you* are deciding, consciously, instead of letting the units decide for you.”

Try it in Python: Standardizing before measuring distances

```
from sklearn.preprocessing import StandardScaler
import numpy as np
X = np.array([[25, 30000], [60, 31000], [26, 45000]]) # age, income
Z = StandardScaler().fit_transform(X)
np.linalg.norm(X[0] - X[1]), np.linalg.norm(X[0] - X[2]) # 1000.6, 15000.0
np.linalg.norm(Z[0] - Z[1]), np.linalg.norm(Z[0] - Z[2]) # 2.16, 2.19
```

...

“And Euclidean distance is only one choice,” Kerguelen went on. “Every clustering method, without exception, rests on some notion of **proximity** between individuals. Choose it badly and no algorithm can save you. So let me show you the family.”

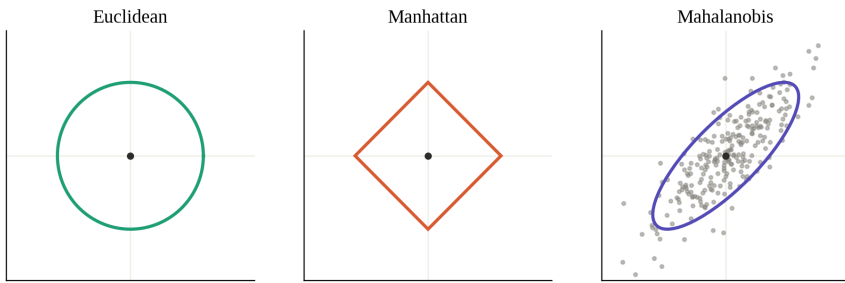
She drew a small square grid, like the streets of Brest’s city centre, rebuilt in straight lines after the war.

“The **Manhattan distance**. Instead of flying straight from one point to another, you walk along the streets: the sum of the absolute differences on each variable. Between two standardized customers who differ by one unit in age and one in income, the Euclidean distance is the square root of two, about 1.41. The Manhattan distance is two. Manhattan is less dominated by a single large difference, because it doesn’t square anything.”

$$\text{Manhattan distance: } d(x, y) = \sum_j |x_j - y_j|$$

“Next, the **Mahalanobis distance**. Suppose two of your variables are strongly correlated, like income and average basket. Rich customers spend more; that’s no surprise. Euclidean distance counts that shared information twice. Mahalanobis uses the covariance matrix Σ of the data to undo the correlation and the differences in scale at the same time. The circles of equal distance become ellipses stretched along the direction where the data naturally varies. A customer who is rich *and* spends a lot is unremarkable. A customer who is poor and spends a lot is unusual, far from everyone else, and Mahalanobis notices.”

$$\text{Mahalanobis distance: } d(x, y) = \sqrt{(x - y)^T \Sigma^{-1} (x - y)}$$



Points at distance 1 from the centre under three distances. Euclidean gives a circle, Manhattan a diamond, and Mahalanobis an ellipse that follows the correlation of the data cloud.

...

“Not every table holds measurements,” Kerguelen continued. “Sometimes it holds counts. Suppose each row is a branch and each column is a type of product, and the cells count how many of each were sold. That’s a *contingency table*. A big branch sells more of everything, but that doesn’t make it different in *profile*. For such tables we use the χ^2 distance — chi-squared — which compares the rows as proportions and weights each column by how rare it is overall.”

$$\chi^2 \text{ distance between rows } i \text{ and } i': d^2(i, i') = \sum_j (1 / f_{\cdot j}) (f_{ij} / f_{i\cdot} - f_{i'j} / f_{i'\cdot})^2$$

“And sometimes a table holds only yes or no. Does this customer have a savings account? A mortgage? The mobile app? For **binary data**, a favourite is the **Jaccard** measure. Look only at the products that at least one of the two customers owns, and ask what fraction they both own.”

She wrote: Anna owns a card, a savings account and the app. Bruno owns a card, a savings account and a mortgage.

“Four products appear between them. They share two. Jaccard similarity: two over four, a half. The Jaccard distance is one minus that: also a half.”

“Why ignore the products neither of them owns?”

“Because we sell forty products. Two customers who both *don’t* own thirty-seven of them aren’t similar in any interesting way. Most people don’t own most things. Jaccard ignores the shared absences on purpose.”

$$\begin{aligned} \text{Jaccard similarity: } J(A, B) &= |A \cap B| / |A \cup B| \\ \text{Jaccard distance: } 1 - J(A, B) \end{aligned}$$

...

“Distance, similarity, dissimilarity — are those the same thing?” asked Gaël.

“Cousins. A **distance** in the strict sense obeys the rules you’d expect: it’s zero only between a point and itself, it’s symmetric, and it obeys the triangle inequality: going from x to z directly is never longer than going via y. A **dissimilarity** is looser: large means different, zero means identical, but the triangle inequality may fail. A **similarity** runs the other way: large means alike. You can always turn one into another.”

“And there’s a stricter kind of distance that will matter later,” Léa said. “The **ultrametric**. It replaces the triangle inequality with something much stronger.”

$$\begin{aligned} \text{triangle inequality: } d(x, z) &\leq d(x, y) + d(y, z) \\ \text{ultrametric inequality: } d(x, z) &\leq \max(d(x, y), d(y, z)) \end{aligned}$$

“In an ultrametric world, every triangle is isosceles, with the two longest sides equal. It sounds exotic, but you already know one. Think of a family tree. How closely related are you to your cousin? As closely as your nearest common ancestor. Your cousin’s brother is exactly as related to you as your cousin is, because you share the same grandparents with both. That’s ultrametric. And when we build trees of clusters, in a few days, the tree will define exactly this kind of distance.”

Kerguelen capped her marker.

“So. The **choice of metric** is a prerequisite for any data analysis. Not a detail. Not a default you leave alone. It decides what ‘similar’ means, and therefore what your clusters will be. Now that we can measure how far apart two customers are, we can talk about what we’re actually trying to build.”

What Gaël learned

Every clustering method relies on a notion of proximity. Euclidean distance is dominated by variables with large units, so standardize (or weight) first. Manhattan sums absolute differences; Mahalanobis corrects for correlation and scale using the covariance matrix; χ^2 compares rows of contingency tables as profiles; Jaccard handles binary data and ignores shared absences. Distances obey the triangle inequality; dissimilarities may not; ultrametrics obey the stronger $d(x,z) \leq \max(d(x,y), d(y,z))$ and are what hierarchical trees produce. Choosing the metric comes first.

Counting Partitions

“What we’re trying to build,” Kerguelen said the next morning, “has a precise name. A **partition**.”

She drew a potato-shaped blob, labelled it Ω , and cut it into four pieces with three wavy lines.

“A partition of Ω is a collection of groups P_1, P_2 , up to P_K , with three properties. Each group is non-empty. No two groups overlap: every customer is in exactly one of them. And together they cover everything: nobody is left out.”

$$\begin{aligned} P_k &\neq \emptyset \\ P_i \cap P_j &= \emptyset \text{ for all } i \neq j \\ P_1 \cup P_2 \cup \dots \cup P_K &= \Omega \end{aligned}$$

“So I just need to find the best partition,” Gaël said. “Try them all, measure how good each one is, keep the winner.”

Kerguelen looked at Léa. Léa looked at the ceiling. It was the look of someone who had once said something very similar.

“How many partitions are there?” Kerguelen asked. “Let’s start small. Ten customers, three groups.”

Gaël guessed. “A few hundred?”

“Nine thousand three hundred and thirty.”

“Oh. Well, a computer can handle that.”

“Twenty-five customers, four groups.”

He shrugged. “A million?”

“Forty-six thousand, seven hundred and seventy-one billion. Roughly 4.7×10^{13} . And a hundred customers into five groups is around 6.6×10^{67} , which is more than the number of atoms in the Earth by a long way.” She wrote the formula on the board. “These are the **Stirling numbers of the second kind**, $S(n, k)$: the number of ways to split n objects into k non-empty groups. They grow roughly like k^n divided by k factorial. You have two hundred thousand customers.”

$$S(n, k) = (1/k!) \sum_{j=0..k} (-1)^j C(k, j) (k - j)^n$$

$$S(10, 3) = 9,330$$

$$S(25, 4) \approx 4.7 \times 10^{13}$$

$$S(100, 5) \approx 6.6 \times 10^{67}$$

Gaël was quiet for a moment. “So trying them all is...”

“Not slow. *Impossible*. Not with every computer ever built, running until the sun burns out. This is called a **combinatorial explosion**, and it’s why every practical clustering algorithm is a **heuristic**. Instead of searching everywhere, we choose a criterion that measures the quality of a partition, start somewhere, and improve step by step. We find a *good* partition, a local optimum. We almost never know whether it’s *the best*.”

...

“There’s one more thing about partitions,” Léa said. “What we just defined is a **hard** partition. Crisp, some people say. Every customer belongs to exactly one group, fully. We can write that as a membership table: a number c_{ik} that is 1 if customer i is in group k , and 0 otherwise.”

“But real customers aren’t always so tidy,” Kerguelen said. “Take Gaël. Student, but he also flies to Lisbon every other month. Is he a ‘student’ or a ‘frequent traveller’?”

“Both,” said Gaël. “Mostly student. My bank balance is very student.”

“Then give him a membership *vector*. Seventy percent student, thirty percent traveller. That’s a **fuzzy partition**. Each c_{ik} is now any number between zero and one, and for each customer the memberships across all the groups add up to one. A hard partition is just the special case where every vector has a single one and zeros everywhere else.”

hard partition: $c_{ik} \in \{0, 1\}$
 fuzzy partition: $c_{ik} \in [0, 1]$ and $\sum_{k=1..K} c_{ik} = 1$ for every i

“When would I want fuzzy?”

“When the groups genuinely blend into each other, like versicolor and virginica. Or when a hard boundary would be misleading, like calling someone at 50.1% ‘definitely a student’. There are fuzzy versions of most algorithms — fuzzy c-means is the best-known cousin of what we’ll see next. But most of the time, in practice, people start with hard partitions. And the most famous way of building one,” she said, standing up, “is also one of the oldest.”

What Gaël learned

A partition splits Ω into K non-empty, non-overlapping groups that cover everything. The number of partitions (Stirling numbers of the second kind) explodes combinatorially, so exhaustive search is impossible and practical algorithms are heuristics that locally optimize a criterion. Hard partitions assign each individual to one class ($c_{ik} \in \{0,1\}$); fuzzy partitions assign membership degrees in $[0,1]$ that sum to 1.

Moving Centres

“The method has three names,” Kerguelen said. “In English, **k-means**. In older French books, *la méthode des centres mobiles* — moving centres — or *réallocation-centrage*, reallocation and centring. The last name is the most honest. It tells you exactly what the algorithm does, over and over, until it stops.”

She wrote it out.

Algorithm 1: k-means

Input: N instances $x_1, \dots, x_N$ in $\mathbb{R}^d$, a number of clusters K

Output: a partition of the instances into K groups

1. Choose K initial centres at random among the instances.
2. Repeat until nothing changes:
 - a. Assign each point to its nearest centre. (reallocation)
 - b. Move each centre to the mean of its class. (centring)

“Two steps. Assign everyone to the nearest centre. Move each centre to the middle of the people assigned to it. Then assign again, because the centres have moved, and some people now have a different nearest centre. Repeat.”

“That’s it?”

“That’s it. The number of clusters K is fixed in advance, you use the Euclidean distance, and every instance normally counts equally, although you can give them weights. It’s simple enough to run by hand. So you will.”

. . .

She gave them seven customers, two variables each, already standardized and shifted to be positive so the arithmetic stayed friendly.

<i>customer</i>	A	B	C	D	E	F	G
<i>x</i>	1	1.5	3	5	3.5	4.5	3.5
<i>y</i>	1	2	3.5	7	5	5	4.5

Seven customers for k-means by hand, $K = 2$.

“K equals two. Léa, you pick A and B as the starting centres. Gaël, you pick A and D.”

Léa went first, working aloud.

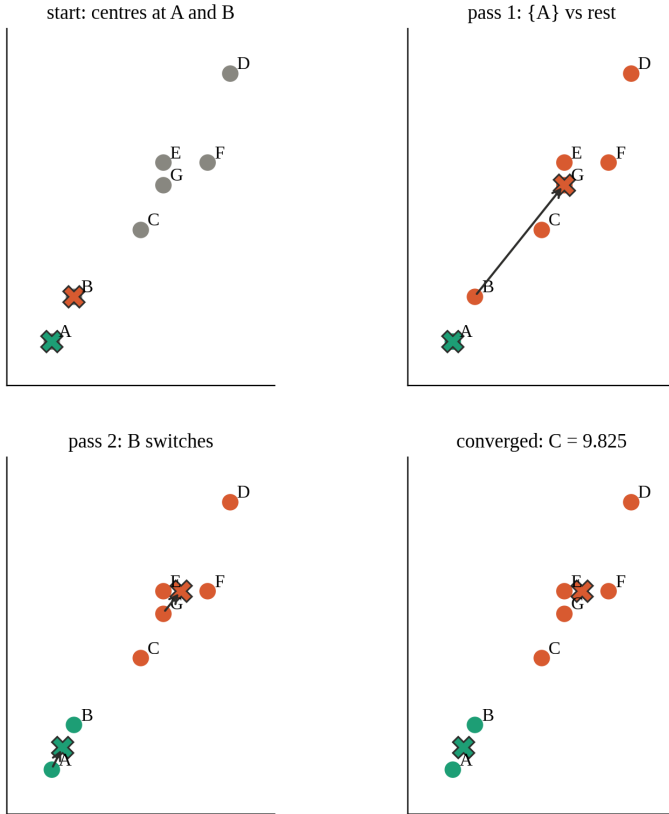
“Centres at A, which is (1, 1), and B, which is (1.5, 2). Step a: every point goes to its nearest centre. A is its own centre. B is its own centre. C, D, E, F, G are all closer to B than to A, because B is a bit further up and to the right. So the groups are {A} and {B, C, D, E, F, G}.”

“Step b.”

“The first centre stays at A. The second moves to the mean of the other six: the x-coordinates add up to 21, divided by six is 3.5; the y-coordinates add up to 27, divided by six is 4.5. New centre at (3.5, 4.5).”

“Again.”

“Now B is closer to A than to (3.5, 4.5). So B switches sides. Groups: {A, B} and {C, D, E, F, G}. New centres: (1.25, 1.5) and (3.9, 5.0). Again... nobody changes group. Done. Three passes.”



Léa's run of k-means on the seven customers. Reading left to right, top to bottom: the initial centres at A and B; the first assignment and the second centre moving to (3.5, 4.5); B switching sides; the final partition. Crosses mark the centres.

Gaël's turn was quicker. "Centres at A (1, 1) and D (5, 7). C, at (3, 3.5), is a squared distance 10.25 from A and 16.25 from D, so it goes with A. Groups: {A, B, C} and {D, E, F, G}. New centres: (1.83, 2.17) and (4.13, 5.38). Assign again... C is now 3.14 from the first centre and 4.78 from the second. Still with A. Nobody moves. Done."

They looked at their two answers. Different.

"Which one is right?" Gaël asked.

. . .

"To answer that," Kerguelen said, "you need to know what k-means is actually trying to do. It has a **quality criterion**. For each class, add up the

squared distances of its points to its centre. Then add up over all classes. That total is the **within-class inertia**. Small inertia means tight classes, points close to their centres.”

$$C(P, L) = \sum_{k=1..K} \sum_{x \in P_k} d^2(x, \lambda_k)$$

$P = (P_1, \dots, P_K)$ a partition, $L = (\lambda_1, \dots, \lambda_K)$ the centres

“Compute it for your two answers.”

Léa: “{A, B} and {C, D, E, F, G}, with their centres: 9.825.”

Gaël: “{A, B, C} and {D, E, F, G}: 10.708.”

“Léa’s is better. Lower inertia. Now, why did k-means stop on Gaël’s partition if a better one exists?”

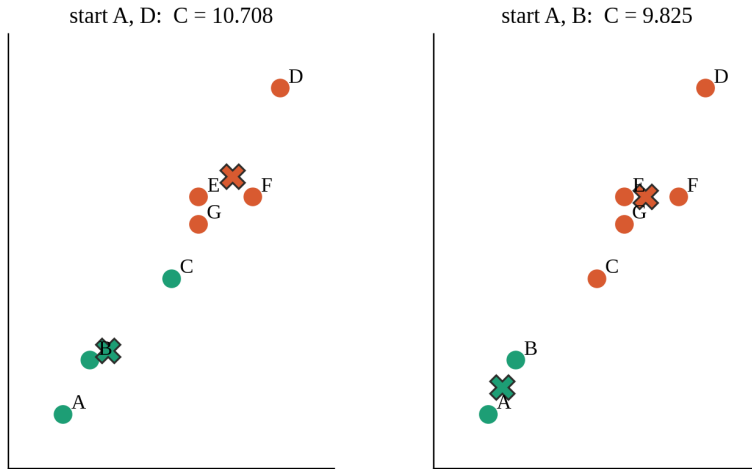
Nobody answered, so she explained.

“Look at what each step does to the criterion. In step a, you keep the centres fixed and move each point to its *nearest* centre. A point’s squared distance to its centre can only go down or stay the same. So the criterion can’t increase. In step b, you keep the groups fixed and move each centre to the mean of its group. And the mean is exactly the point that minimizes the sum of squared distances to a set of points. Take the derivative of the sum of $(x - \lambda)^2$ with respect to λ , set it to zero, and you get λ equal to the average. So step b can’t increase the criterion either.”

$$\begin{aligned} \frac{\partial}{\partial \lambda} \sum_{x \in P_k} \|x - \lambda\|^2 &= -2 \sum_{x \in P_k} (x - \lambda) = 0 \\ \Rightarrow \\ \lambda &= (1 / |P_k|) \sum_{x \in P_k} x \end{aligned}$$

“So the criterion goes down at every step,” Léa said. “In my run: 83.0 with the starting centres, then 21.5, then 12.5, then 9.825.”

“And there are only finitely many partitions, so it can’t go down forever. It *must* stop. That’s the **convergence** guarantee. But notice what it doesn’t promise. It stops when no single step can improve things. That’s a **local optimum**: a valley you can’t walk out of by taking small steps downhill, even though a deeper valley may exist over the next hill. Gaël started in the wrong valley. His answer is stable. It’s just not the best.”



Same data, two starting points, two stable answers. Gaël's run (left) stops with within-class inertia 10.708; Léa's run (right) reaches 9.825. Both are fixed points of *k*-means; only one is better.

“So what do people do?”

“Run it many times from different random starts and keep the partition with the lowest inertia. Every good library does that for you. Many also use a smarter starting rule called **k-means++**, which spreads the initial centres out instead of choosing them blindly. That’s not in your lecture slides, but you’ll meet it the moment you open the documentation. Each run is cheap: one pass costs about $N \times K \times d$ distance calculations.”

Try it in Python: k-means in scikit-learn

```
import numpy as np
from sklearn.cluster import KMeans
X = np.array([[1,1],[1.5,2],[3,3.5],[5,7],[3.5,5],[4.5,5],[3.5,4.5]])
KMeans(n_clusters=2, init=np.array([[1,1],[5,7]]), n_init=1).fit(X).inertia_ # 10.708
KMeans(n_clusters=2, init=np.array([[1,1],[1.5,2]]), n_init=1).fit(X).inertia_ # 9.825
km = KMeans(n_clusters=2, n_init=10, random_state=0).fit(X) # 10 random restarts
km.labels_, km.cluster_centers_, km.inertia_ # keeps the best: 9.825
```

...

“It’s elegant,” Gaël said. “Why doesn’t everyone just use k-means for everything?”

“Because it has weaknesses, and they’re typical of clustering in general.” Kerguelen counted them off.

“One: you must choose **K in advance**. The algorithm will happily cut your customers into seven groups if you ask for seven, whether or not seven groups exist. Two: **local optimization**. It produces a sequence of partitions and centres whose criterion keeps decreasing, but it can get stuck, as you just saw. Three: **sensitivity to outliers and extreme values**. The centre is a mean, and a mean is easily dragged. Put one billionaire in a cluster of students, and the ‘centre’ of the student group is suddenly a rich person.”

“And a fourth that follows from the criterion,” Léa added. “Because it minimizes squared distances to a centre, it likes round, compact clusters of similar size. Give it long, curved or nested shapes and it will slice them into pieces that look nothing like the real structure.”

“All of which,” said Kerguelen, “brings us to the medoid.”

What Gaël learned

k-means (moving centres, reallocation–centring) fixes K , starts from K random centres, then alternates: assign each point to its nearest centre; move each centre to its class mean. It minimizes the within-class inertia $C(P, L) = \sum_k \sum_{x \in P_k} d^2(x, \lambda_k)$. Both steps can only lower C , and there are finitely many partitions, so it converges, but only to a local optimum; run it from several starts. Weaknesses: K chosen in advance, local optima, sensitivity to outliers, and a preference for compact, roughly spherical clusters.

The Medoid

Gaël came in the next morning looking pleased and slightly suspicious of being pleased.

“I ran k-means on the real data. Five clusters, a hundred restarts, kept the best. One cluster is weird, though. It’s supposed to be ‘young, low income’, but its centre has an average income of ninety thousand euros.”

Léa pulled up the cluster. Four thousand students and young workers, and one customer with an income of three hundred million euros a year, who for reasons of his own had a student account.

“There’s your ninety thousand,” she said.

Kerguelen wrote five numbers on the board: 1, 2, 3, 4, 100.

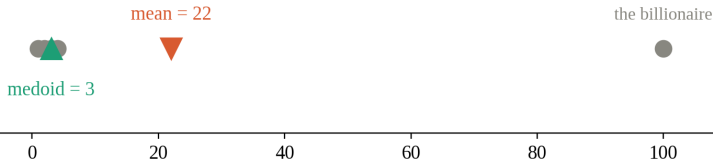
“What’s the centre of this group?”

“The mean is 22,” Gaël said. “Which is nowhere near any of them.”

“The mean belongs to nobody. Now choose instead the *point of the group itself* whose total distance to the others is smallest.”

Gaël calculated. For 3: distances 2, 1, 0, 1, 97, total 101. For 2 and 4 the totals were higher. For 100, much higher. “It’s 3.”

“That’s a **medoid**: the most central *actual member* of a group. It can’t be dragged into empty space, because it must be one of the points. The billionaire can pull a mean anywhere he likes. He can barely move a medoid.”



One extreme value drags the mean (the k-means centre) far away from the group, while the medoid stays on a real, central point.

...

“So we replace means with medoids,” Kerguelen said. “The best-known method is **Partitioning Around Medoids**, PAM, by Kaufman and Rousseeuw. Instead of moving centres around in space, it keeps K real customers as representatives and tries swapping them.”

Algorithm 2: PAM

Input: N instances $x_1, \dots, x_N$, a dissimilarity d , a number of clusters K

Output: a partition into K groups, each represented by a medoid

1. Choose K initial medoids at random among the instances.
2. Assign every point to its nearest medoid; the cost is the sum of the dissimilarities from each point to its medoid.
3. For each medoid m and each non-medoid point b :
try swapping m and b , and recompute the total cost.
4. Keep the configuration with the lowest cost.
5. Repeat steps 2 to 4 until the medoids no longer change.

“It’s slower than k-means,” Léa said. “Each round tries every medoid against every other point: about $K \times (N - K)$ swaps, and each swap needs the cost recomputed over all $N - K$ non-medoids. Roughly $K(N - K)^2$ operations per round.”

“Much slower,” Kerguelen agreed. “On two hundred thousand customers, PAM itself would take a very long time; people use sampling variants such as CLARA for large data. But look at what you gain. First, robustness: outliers barely move the medoids. Second, **any dissimilarity works**. k-means needs means, so it needs Euclidean space. PAM only ever compares distances between real points. Jaccard, χ^2 , anything. Third, every cluster is represented by a real customer, which marketing loves. You can show them ‘Nathalie, 34, two children, fixed-rate mortgage’ instead of ‘the

centroid’.”

$$\text{cost}(M) = \sum_{i=1..N} \min_{m \in M} d(x_i, m)$$

M = the set of K medoids

Try it in Python: A compact PAM in NumPy

```
import numpy as np
def pam(D, K, seed=0):
    # D: N x N dissimilarity matrix
    rng = np.random.default_rng(seed)
    med = list(rng.choice(len(D), K, replace=False))
    cost = D[:, med].min(1).sum()
    while True:
        best = (cost, med)
        for i in range(K):
            # each medoid...
            for b in range(len(D)):
                # ...against each non-medoid
                if b in med: continue
                trial = med[:i] + [b] + med[i+1:]
                c = D[:, trial].min(1).sum()
                if c < best[0]: best = (c, trial)
        if best[0] >= cost: break
        cost, med = best
    return med, D[:, med].argmin(1), cost # medoids, labels, cost
```

Kerguelen showed them a classic test set, the Ruspini data: seventy-five points in the plane that any human would split into four groups at a glance. PAM with K equal to four found exactly those four groups, each with a medoid sitting at its heart.

...

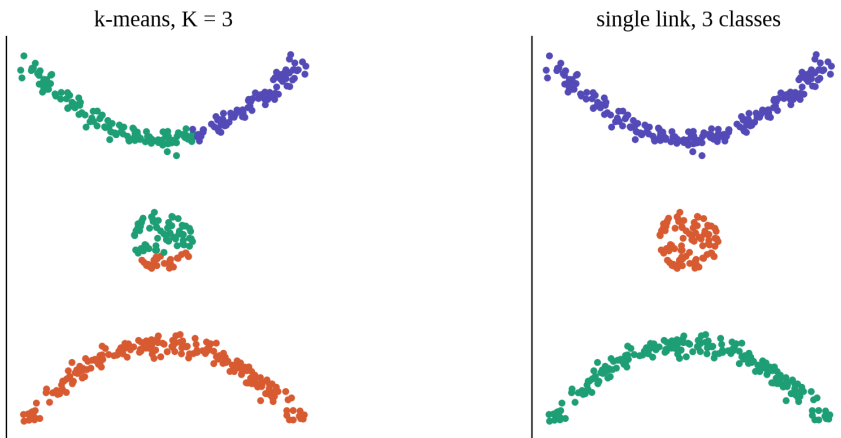
“Then PAM fixes everything,” Gaël said.

“PAM fixes the outlier problem. It doesn’t fix the shape problem.” Kerguelen opened another dataset. Two long, curved bands, like a pair of eyebrows, one above the other, and a small round blob between them. “This is called the Cassini problem. Three groups, obvious to the eye. Run k -means or PAM with K equal to three.”

Both methods did the same thing: they cut the bands in half, joining the end of one eyebrow with the blob or with a piece of the other eyebrow, because that’s what makes compact, round groups.

“Both methods think in terms of a *centre*. Every cluster is ‘the points closest to this representative’. So their clusters are always convex blobs

around a point. If the true groups are long, thin or curved, no centre can describe them.”



Two curved bands and a small blob. Left: a centre-based method such as k-means cuts the bands into compact pieces. Right: single-link hierarchical clustering follows the chains of close neighbours and recovers the three shapes.

“And the picture on the right?”

“That,” said Kerguelen, “is what happens when you stop thinking about centres, and start thinking about who is next to whom. Tomorrow we’re going to build a tree.”

What Gaël learned

A medoid is the most central actual member of a group. PAM (Partitioning Around Medoids) keeps K real points as representatives and repeatedly tries swapping a medoid with a non-medoid, keeping any swap that lowers the total dissimilarity. It is robust to outliers, works with any dissimilarity, and yields interpretable representatives, but costs about $K(N-K)^2$ per round. Like k-means it still needs K and still builds compact clusters around centres, so it fails on long or curved shapes.

The Family Tree

“Yesterday,” Kerguelen said, “you asked me for K groups and I gave you K groups. Today we won’t choose K at all. We’ll build every partition at once, from the finest to the coarsest, nested inside each other like Russian dolls.”

“An **indexed hierarchy**,” Léa told Gaël, “is a sequence of nested partitions. At the bottom, every customer is alone in his own group. At the top, everybody is in one single group. In between, each level is obtained from the one below by merging groups, never by splitting them. We draw it as a tree, a **dendrogram**. And each merge happens at a certain height, which tells you how different the two groups were when they were joined.”

“There are two ways to grow such a tree,” Kerguelen said. “**Divisive**, or top-down: start with everybody in one group and keep splitting until everybody is alone. And **agglomerative**, or bottom-up: start with everybody alone and keep merging the two most similar groups until only one is left. The second is far more common, and it has its own name, **agglomerative hierarchical clustering**, AHC. In French, CAH.”

Algorithm: agglomerative hierarchical clustering

Input: an $N \times N$ dissimilarity matrix, a linkage rule $D(A, B)$

Output: a dendrogram: $N - 1$ nested merges and their heights

1. Start with N singleton classes $\{x_1\}, \dots, \{x_N\}$.
2. Repeat $N - 1$ times:
 - a. Find the two classes A, B with the smallest $D(A, B)$.
 - b. Merge them into $A \cup B$, recording the height $D(A, B)$.
 - c. Update the dissimilarities between $A \cup B$ and every other class.

...

“Step a hides the only real question,” Kerguelen said. “We know the distance between two *customers*. But what is the distance between two *groups* of customers? That’s the **linkage** rule, or aggregation criterion, and different rules build very different trees.”

She wrote six customers on a line — perhaps their scores on some spending index — and named them a to f: 1, 2, 4, 7.5, 9 and 15.

“**Single link**, also called minimum link: the distance between two groups is the distance between their *closest* members. Two groups are near if *anyone* in one is near *anyone* in the other.”

$$\text{single link: } D(A, B) = \min_{x \in A, y \in B} d(x, y)$$

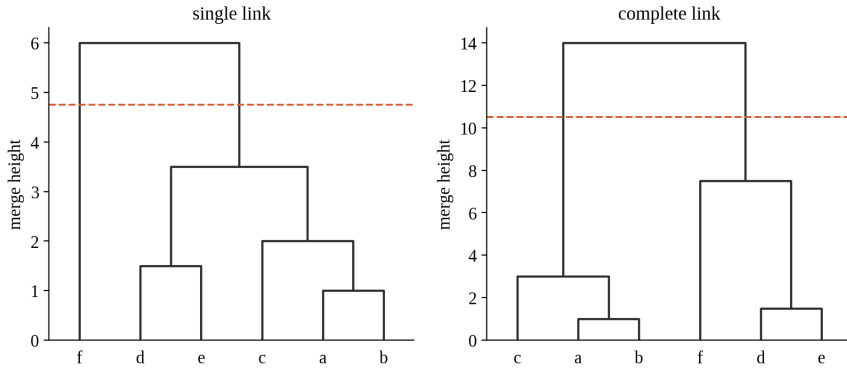
Léa ran it on the board. “a and b are 1 apart: merge at height 1. d and e are 1.5 apart: merge at 1.5. c is 2 from b, so c joins {a, b} at height 2. Now {a, b, c} and {d, e}: closest members are c at 4 and d at 7.5, distance 3.5. And f is 6 from e. So {a, b, c} and {d, e} merge at 3.5, and f joins last, at 6.”

“**Complete link**, or maximum link: the distance between two groups is the distance between their *farthest* members. Two groups are near only if *everyone* is near *everyone*.”

$$\text{complete link: } D(A, B) = \max_{x \in A, y \in B} d(x, y)$$

“Same first two merges, at 1 and 1.5. Then c joins {a, b}, but at height 3 now, its distance to a. Then... {a, b, c} to {d, e}: farthest members a and e, 8 apart. f to {d, e}: farthest pair f and d, 7.5 apart. So this time f joins {d, e} first, at 7.5, and the two halves meet at the top at 14.”

“Two rules, two different trees. Cut both trees into two groups and you get two different answers: single link says {a, b, c, d, e} and {f}; complete link says {a, b, c} and {d, e, f}.”



Dendrograms for the six customers. Single link (left) chains d and e onto the left group and leaves f alone. Complete link (right) builds two balanced halves. The dashed line cuts each tree into two classes.

“Single link has a famous personality,” Kerguelen said. “It *chains*. If there’s a path of close neighbours from one point to another, single link will eventually join them, however long and thin the path. That’s exactly why it found the curved eyebrows yesterday. It’s also why, on noisy data, it sometimes produces one giant snake and a few loners. Complete link does the opposite: it likes compact groups of similar diameter, and resists chaining.”

“And in between, **average link**: the average distance over all pairs, one from each group. A compromise between the two.”

$$\text{average link: } D(A, B) = \left(\sum_{x \in A} \sum_{y \in B} d(x, y) \right) / (|A| \cdot |B|)$$

...

“And then there’s the criterion most people end up using,” Kerguelen said, “**Ward’s criterion**. It brings us back to inertia.”

She drew two small clouds with their centres.

“The total inertia of the data — the sum of squared distances to the overall centre — splits into two parts: the **within-class** inertia, how spread out each group is around its own centre, and the **between-class** inertia,

how spread out the group centres are around the overall centre. The total never changes. So every time you merge two groups, within-class inertia goes up and between-class inertia goes down by exactly the same amount. Ward says: at each step, merge the pair of groups that *increases the within-class inertia the least*. And that increase has a neat closed form.”

$$\text{Ward: } D(A, B) = \left(\frac{n_A n_B}{n_A + n_B} \right) \cdot d^2(g(A), g(B))$$

n_J = size (or weight) of J, $g(J)$ = centroid of J

“Two effects in one formula. The squared distance between the centres, so far-apart groups are expensive to merge. And the factor $n_A n_B / (n_A + n_B)$, which grows with the sizes, so two big groups are expensive to merge. On our six customers: a and b cost a half. d and e cost 1.125. Then c with {a, b}, whose centre is 1.5, costs two-thirds of 2.5 squared, about 4.17. At the end, f with {d, e} costs about 30.4, while {a, b, c} with {d, e} would cost 42. So Ward, like complete link, sends f to {d, e}.”

“It’s k-means’ criterion,” Léa said slowly. “Within-class inertia. Just optimized greedily, one merge at a time, instead of by moving centres.”

“Exactly. Which is why Ward and k-means are often used together. Build a Ward tree to decide how many groups look reasonable, then use its partition to start k-means and polish it.”

Try it in Python: Hierarchical clustering with SciPy

```
import numpy as np
from scipy.cluster.hierarchy import linkage, fcluster, dendrogram
x = np.array([1, 2, 4, 7.5, 9, 15]).reshape(-1, 1)
for m in ("single", "complete", "average", "ward"):
    Z = linkage(x, method=m) # the whole tree
    print(m, fcluster(Z, t=2, criterion="maxclust"))
# single [1 1 1 1 1 2] complete [1 1 1 2 2 2]
# average [1 1 1 1 1 2] ward [1 1 1 2 2 2]
dendrogram(Z) # draw it (with matplotlib)
```

...

“Where does the ultrametric come in?” Gaël asked. “You promised.”

“Take any two customers and look at the height at which they first end up in the same group. That height defines a new distance between them,

sometimes called the cophenetic distance. And it's always an ultrametric: two customers are exactly as far apart as the merge that joined their branches, just like cousins and their common grandparents. Building the hierarchy amounts to approximating your original dissimilarity by an ultrametric, and each linkage rule makes that approximation in its own way."

"One warning," Léa added. "AHC starts from the full $N \times N$ dissimilarity matrix. For two hundred thousand customers that's twenty billion distances, before you've merged anything. It's wonderful for thousands of points and for understanding structure. For hundreds of thousands, people cluster a sample, or run k-means first into a few hundred small groups and build the tree on those."

"So now we have a tree," Gaël said. "Where do I cut it?"

Kerguelen smiled. "That," she said, "is the only question that really matters."

What Gaël learned

Hierarchical clustering builds nested partitions shown as a dendrogram, either divisive (top-down) or agglomerative (bottom-up, AHC). At each step AHC merges the two closest classes according to a linkage: single (min, chains, finds elongated shapes), complete (max, compact groups), average (mean of pairwise distances), or Ward, $D(A,B) = n_A n_B / (n_A + n_B) \cdot d^2(g(A), g(B))$, which merges the pair that least increases within-class inertia. Merge heights define an ultrametric. AHC needs the $N \times N$ matrix, so it suits thousands of points, not millions.

How Many?

“There are two questions you can’t avoid,” Kerguelen said. “How many classes, K ? And is the partition you found any good? In supervised learning you had a test set with the right answers. Here there are no right answers to test against. So how do you validate?”

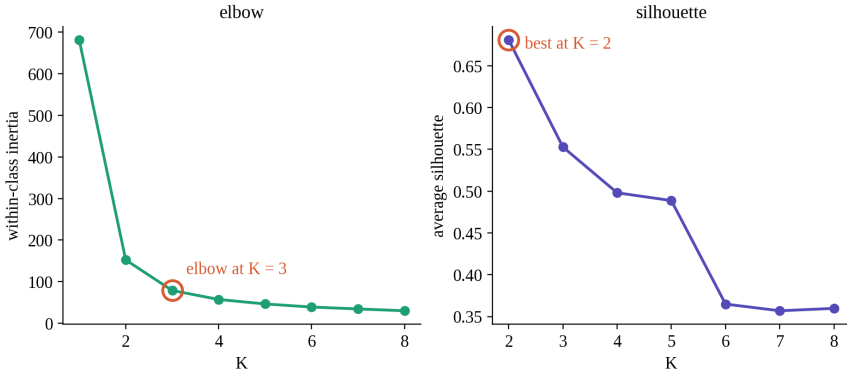
“Broadly, two ways,” Léa said. “The first is **expert validation**. You choose K and a partition, and you show it to people who know the domain. Do these groups mean something? Can marketing name them? Would a different offer really suit each one? It’s subjective, and it’s also the most important test there is, because a partition nobody can use is worthless, however good its numbers.”

“The second is **model-based selection**. You run your methods for several values of K , maybe several methods, and you compare them with a numerical criterion. The oldest trick is the **elbow**.”

. . .

She ran k-means on Fisher’s irises for K from 1 to 8, all four measurements, and plotted the within-class inertia.

“Inertia always decreases as K grows. With K equal to N , every point is its own centre and inertia is zero. So you can’t just minimize it. You look for the point where adding one more cluster stops paying off: where the curve bends like an elbow. Here it falls from 152 at $K = 2$ to 79 at $K = 3$, then only to 57, 46, 39. The big gain stops after three. The elbow says three.”



k-means on Fisher's irises (four measurements). Left: within-class inertia keeps falling, with an elbow at $K = 3$. Right: the average silhouette is highest at $K = 2$.

“That’s a little vague,” Gaël said. “Where it bends.”

“It is vague. So people have built a whole research field of **partition quality criteria**.” Kerguelen listed three families.

“One: **isolation and connectivity**, from Pauwels and Frederix. Good clusters are well separated from each other, and inside each cluster, points are linked to their neighbours. Two: **stability**. Draw bootstrap samples — resample your customers with replacement — cluster each sample, and see whether you keep finding the same groups. Real structure is stable. Groups that come and go with each resample were probably noise. Three: **internal indices** built from within-class and between-class inertia or distances. Dunn, Davies–Bouldin, the silhouette, the W/B ratio, the gap statistic, and many more.”

$$\text{Dunn} = \min_{k \neq l} \delta(P_k, P_l) / \max_k \text{diam}(P_k)$$

(higher is better)

$$\text{Davies–Bouldin} = (1/K) \sum_k \max_{l \neq k} (s_k + s_l) / d(g_k, g_l)$$

(lower is better; s_k = mean distance of P_k to its centroid)

“Dunn compares the smallest gap between two clusters with the widest cluster. Davies–Bouldin looks, for each cluster, at its worst neighbour: how spread out the two are compared with how far apart their centres sit. But the one you’ll use most often is the silhouette. Let’s do it by hand.”

. . .

She wrote five points on a line and split them into two clusters: {1, 2, 3} and {8, 9}.

“For each point i , first compute a_i , its average distance to the *other* members of its own cluster. How well does it fit where it is? Then compute its average distance to each *other* cluster, and call the smallest of those b_i . How close is the next-best home? The **silhouette** of i compares the two.”

$$\begin{aligned} d_{i,C} &= (1/|C|) \sum_{x_t \in C} d(x_i, x_t) \\ a_i &= d_{i,C(i)} \\ b_i &= \min_{C \neq C(i)} d_{i,C} \end{aligned}$$

$$\begin{aligned} \text{Sil}_i &= (b_i - a_i) / \max(a_i, b_i) \\ S &= (1/N) \sum_{i=1..N} \text{Sil}_i \end{aligned}$$

“Take the point 3. Its cluster-mates are 1 and 2, at distances 2 and 1: so a is 1.5. The other cluster, 8 and 9, is at distances 5 and 6: so b is 5.5. The silhouette is $(5.5 - 1.5) / 5.5$, about 0.73.”

Gaël did the rest. “For 1: a is 1.5, b is 7.5, silhouette 0.8. For 2: a is 1, b is 6.5, silhouette about 0.85. For 8: a is 1, b is 6, about 0.83. For 9: a is 1, b is 7, about 0.86. Average: 0.81.”

“Read it like this. A silhouette near 1: the point sits snugly in its cluster, far from any other. Near 0: it’s on the border between two clusters. Negative: it’s closer, on average, to another cluster than to its own. It’s probably in the wrong place. Point 3 has the lowest score because it’s the one nearest the other group. And the average silhouette S of the whole partition is a single number you can compare across values of K . You choose the K that **maximizes** it.”

Try it in Python: Elbow and silhouette on the iris data

```

from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
X = load_iris().data
for k in range(2, 7):
    km = KMeans(n_clusters=k, n_init=10, random_state=0).fit(X)
    sil = silhouette_score(X, km.labels_)
    print(k, round(km.inertia_, 1), round(sil, 3))
# 2 152.3 0.681    3 78.9 0.553    4 57.2 0.498
# 5 46.4 0.489    6 39.0 0.365

```

...

Léa ran it on the irises. Gaël read out the silhouettes: 0.681 for $K = 2$. 0.553 for $K = 3$. Then lower and lower.

“The silhouette says two,” he said. “The elbow says three. The biologists say three species. Who’s right?”

Kerguelen looked genuinely delighted, which Gaël was learning to distrust.

“All of them. The silhouette is right that the data shows two very clear groups: setosa, and everything else. Versicolor and virginica touch; their border points drag the silhouette down when you separate them. The elbow is right that separating them still reduces inertia a lot. And the biologists are right about the species, but they know things that aren’t in these four measurements, such as which flowers can breed together.” She shrugged. “This is the picture I promised we’d come back to. The criteria don’t hand you the truth. They’re evidence. You weigh them, you look at the groups, you ask the experts, and you decide. That’s why validation is a research field, and not a button.”

“So for the customers...”

“Compute several criteria. Check stability with bootstrap samples. Look at the dendrogram for big jumps in merge height. Then sit down with marketing and see which partition they can actually do something with.”

It took him a week. The answer was six.

What Gaël learned

The two key questions are choosing K and validating the partition. Validation is either expert-based (do the groups mean something useful?) or model-based (compare several K or methods with a criterion). The elbow method looks for where within-class inertia stops falling fast. Other approaches: isolation and connectivity; stability under bootstrap resampling; internal indices such as Dunn (higher is better), Davies–Bouldin (lower is better), W/B ratio, gap and the silhouette, $Sil_i = (b_i - a_i) / \max(a_i, b_i)$, averaged and maximized. Criteria can disagree (irises: elbow 3, silhouette 2), so they are evidence, not verdicts.

Salmon and Sea Bass

The segmentation went to marketing on a Friday. On Monday, the director came down with a new problem.

“Marketing loves the six groups,” Jean-Yves Le Bihan said. “Now they want to know, for every *new* customer who opens an account, which of the six groups they belong to. On day one, before we have months of their spending.”

When he’d gone, Gaël frowned. “Isn’t that the same problem?”

“No,” said Léa. “It’s the opposite problem. Last month nobody had labels, and we invented the groups. Now every existing customer *has* a label — the group we gave them — and we want to learn a rule that predicts the label for someone new. That’s **supervised learning**. The whole difference between the two is one question: *is class information available for the individuals?*”

“In supervised learning,” Kerguelen said, “every example comes with its label, and we learn a **decision rule** — also called a classifier, a model, a hypothesis — to predict the labels of new observations. Geometrically, you can picture the rule as a surface in the space of variables, a *hypersurface* separating the examples of one class from those of another. And how hard the problem is depends on what you’re predicting.”

$Y \in \mathbb{R}^d$: (multiple) regression
 $Y \in \{-1, +1\}$: two-class discrimination
 $Y \in \{1, \dots, q\}$: multi-class discrimination

“Regression, when the target is a number, like next month’s spending. Two-class discrimination, like fraud or not fraud. Multi-class discrimination, like our six groups.”

. . .

“There’s a story every textbook tells,” Kerguelen said, “and since we’re in Brest, you should hear it. A fish-processing company wants to sort fish automatically on a conveyor belt, using cameras. A pilot study on two species: salmon and sea bass. They photograph a sample of fish, and a person labels each one.”

“So the labels come from the person.”

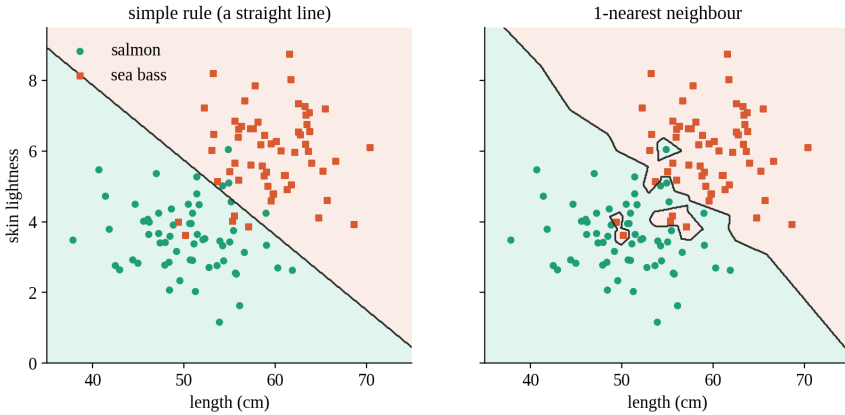
“Yes. Then comes **feature selection**: looking at the first images to decide which measurements might tell the species apart. Length? Width? Number of fins? Skin brightness? In the pilot study, two stand out. Length, and the lightness of the skin. Sea bass tend to be longer and lighter; salmon shorter and darker. Not always. There’s overlap.”

She drew a scatter plot, length across, lightness up, the two species in two colours overlapping in the middle.

“We have a **training set**, and on it we build a decision rule. A straight line, perhaps. Or a curve that wriggles between the points to get every training fish right. The rule depends on the method, and it can be more or less refined.”

Gaël, who had been through Léa’s lessons on bias and variance, said it before she could. “And the wriggly one will do worse on new fish.”

“It usually does. The rule is then tested on **new examples**, fish it never saw, and we measure the **error rate**, the misclassification rate, on that new data. That’s generalization. The methodology is the one Léa learned last year: split your data into a **training** set, a **validation** set for choosing settings, and a **test** set, used once at the end, or use cross-validation. And look for **parsimony**: Occam’s razor. As the model gets richer, its error on the training data keeps falling, but its real error on new data falls and then rises. Too simple: high bias, underfitting. Too rich: high variance, overfitting. The ideal is the compromise in the middle.”



Salmon and sea bass described by length and skin lightness. Left: a simple, straight decision boundary. Right: a 1-nearest-neighbour boundary that bends around every training fish, a typical sign of overfitting.

...

“Now,” Kerguelen said, “let me show you the simplest supervised algorithm there is. You’ll recognize its heart, because it’s the same thing that powers clustering: a distance.”

Algorithm 3: k-nearest neighbours (kNN)

Input: N labelled instances (x_i, y_i) , a distance d , an integer $k \geq 1$
 Output: a label for each new observation x

1. Compute $d(x, x_i)$ for every training instance i .
2. Sort the distances in ascending order.
3. Predict the label of x by majority vote among its k nearest neighbours.

“That’s all. No training at all, really. You just remember every example, and when a new one arrives you ask its neighbours.”

She gave them seven fish and a new one: 55 centimetres long, lightness 4.5.

<i>length (cm)</i>	<i>lightness</i>	<i>species</i>	<i>distance to new fish</i>
48	3.1	salmon	7.14
52	3.8	salmon	3.08
60	4.0	salmon	5.02
45	2.5	salmon	10.20
58	6.2	sea bass	3.45
54	5.9	sea bass	1.72
66	7.0	sea bass	11.28
55	4.5	?	—

Seven labelled fish and a new one. Euclidean distances on the raw measurements.

Gaël computed the distances. “Nearest is the sea bass at 54 cm, distance 1.72. Then a salmon, 3.08. Then a sea bass, 3.45. Then two salmon, 5.02 and 7.14.”

“k equal to 1?”

“Sea bass. Only the nearest neighbour votes.”

“k equal to 3?”

“Two sea bass, one salmon. Sea bass.”

“k equal to 5?”

He counted. “Three salmon, two sea bass.” He looked up. “Salmon. The same fish changes species depending on k.”

“So k is the complexity dial. With k equal to one, the rule follows every single training point, including the noisy ones. That’s the wriggly boundary: low bias, high variance. With a huge k, the rule becomes very smooth; in the extreme, k equal to N, it predicts the majority class for everybody. High bias. And you choose k the way you choose any setting: by cross-validation.”

“And be careful with the k’s,” Léa said. “The k in k-means is the number of *clusters*. The k in kNN is the number of *neighbours*. Same letter, completely different jobs.”

“And look at the distances again,” Kerguelen added. “Length is measured in centimetres and ranges over twenty units. Lightness ranges over five. Just like Anna, Bruno and Chloé, the variable with the bigger numbers dominates. So you standardize here too. Everything you learned

about distances in the first week applies to kNN just the same.”

Try it in Python: kNN, with scaling and cross-validation

```
import numpy as np
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import cross_val_score
from sklearn.datasets import load_iris
Xf = np.array([[48,3.1],[52,3.8],[60,4.0],[45,2.5],[58,6.2],[54,5.9],[66,7.0]])
yf = np.array(["salmon"]*4 + ["sea bass"]*3)
for k in (1, 3, 5):
    print(k, KNeighborsClassifier(n_neighbors=k).fit(Xf, yf).predict([[55, 4.5]]))
# 1 sea bass    3 sea bass    5 salmon
X, y = load_iris(return_X_y=True)
for k in (1, 5, 15, 50):
    # choose k by 10-fold CV
    model = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=k))
    print(k, cross_val_score(model, X, y, cv=10).mean()).round(3))
# 1 0.953    5 0.953    15 0.96    50 0.88
```

...

Gaël built the new-customer classifier over the following weeks. The labels were the six segments from the clustering. The features were only what the bank knew on day one. He tried kNN with scaled features, a logistic regression and a small forest, compared them by stratified cross-validation, and chose the forest, which was about as accurate as anything else and much quicker at prediction time on two hundred thousand customers.

“It’s strange,” he said to Kerguelen one evening. “We used unsupervised learning to *create* the labels, and then supervised learning to *predict* them.”

“It’s very common. And it’s a good way to remember the difference. Unsupervised learning asks: *what structure is there?* Supervised learning asks: *given the structure we know, what is this new thing?* One explores. The other decides. In both, the heart of the matter is the same: what does it mean for two things to be alike?”

What Gaël learned

The difference between supervised and unsupervised learning is whether class labels are available. Supervised learning learns a decision rule (geometrically, a separating hypersurface) to predict labels: regression if $Y \in \mathbb{R}^q$, two-class or multi-class discrimination otherwise. The workflow: choose discriminant features (salmon vs sea bass: length and lightness), build the rule on a training set, measure the error rate on new data, split into train/validation/test (or cross-validate), and prefer parsimonious models. kNN predicts by majority vote among the k nearest training points; k controls complexity (small k overfits), should be chosen by cross-validation, and requires scaled features.

Epilogue

Six Groups

In January, marketing named the segments. They called them, in order of size: the *Steady Savers*, the *Young and Online*, the *Busy Families*, the *Weekend Travellers*, the *Quiet Retirees* and — the smallest, eleven hundred customers, found only by the Ward tree and confirmed by every bootstrap sample — the *Fishermen*. They were exactly that. Boat owners from the ports of Finistère, with irregular, seasonal incomes, high fuel spending in spring and almost nothing in winter. Nobody upstairs had known they were a group. The bank had been sending them credit card offers designed for commuters.

“That’s the one I’m proudest of,” Gaël said. “The one nobody expected.”

“That’s the one clustering is *for*,” said Kerguelen.

She had brought a present: a small, worn book, a field guide to the fishes of the Atlantic coast. On the inside cover she had written a single line.

Similar to what? Measured how? Grouped by whom? — Ask these three questions before any algorithm.

Léa read it over his shoulder and smiled. It was, she thought, the unsupervised version of *ninety-nine percent of what?*

Cheat Sheet

Concept	Meaning	Ch.
Clustering	Organize N individuals into homogeneous, natural classes without labels or a priori class definitions.	1
Data	$N \times P$ table: individuals $\Omega = \{x_1, \dots, x_N\}$, P variables.	1
Euclidean	$\sqrt{\sum_j (x_j - y_j)^2}$. Standardize variables first.	2
Manhattan	$\sum_j x_j - y_j $.	2
Mahalanobis	$\sqrt{((x - y)^T \Sigma^{-1} (x - y))}$; corrects for scale and correlation.	2
χ^2 distance	Compares rows of a contingency table as profiles, weighting columns by $1/f_{\cdot j}$.	2
Jaccard	$1 - A \cap B / A \cup B $, for binary data; ignores shared absences.	2
Ultrametric	$d(x, z) \leq \max(d(x, y), d(y, z))$; defined by dendrogram merge heights.	2, 6
Partition	Non-empty $P_1, \dots, P_K$, pairwise disjoint, union = Ω .	3
Number of partitions	Stirling number $S(n, k) \approx k^n/k!$; $S(25, 4) \approx 4.7 \times 10^{13}$. Hence heuristics.	3
Hard vs fuzzy	$c_{ik} \in \{0, 1\}$ vs $c_{ik} \in [0, 1]$ with $\sum_k c_{ik} = 1$.	3
k-means	Choose K centres; repeat: assign to nearest centre, move centres to class means.	4

Concept	Meaning	Ch.
Within-class inertia	$C(P, L) = \sum_k \sum_{x \in p_k} d^2(x, \lambda_k)$; decreases at every step; local optimum ^k only.	4
k-means weaknesses	K fixed in advance, local optima (use restarts, k-means++), outliers, compact convex shapes only.	4
Medoid	Most central actual member of a group; robust to outliers.	5
PAM	Swap medoids with non-medoids while the total dissimilarity decreases; any dissimilarity; $\sim K(N-K)^2$ per round.	5
AHC	Bottom-up merging of the two closest classes; result is a dendrogram (divisive = top-down).	6
Single link	$\min_{x \in A, y \in B} d(x, y)$; chains; finds elongated shapes.	6
Complete link	$\max_{x \in A, y \in B} d(x, y)$; compact groups.	6
Average link	$\frac{\sum_{x \in A} \sum_{y \in B} d(x, y)}{(A B)}$.	6
Ward	$n_A n_B / (n_A + n_B) \cdot d^2(g(A), g(B))$; smallest increase of within-class inertia.	6
Choosing K	Expert validation; elbow of inertia; stability under bootstrap; internal indices.	7
Dunn / Davies–Bouldin	Min separation / max diameter (higher better); mean worst-neighbour ratio (lower better).	7
Silhouette	$Sil_i = (b_i - a_i) / \max(a_i, b_i) \in [-1, 1]$; maximize the average S.	7
Supervised learning	Labels available; learn a decision rule (a separating hypersurface). Regression, two-class, multi-class.	8
kNN	Majority vote of the k nearest labelled points; small k overfits; choose k by cross-validation; scale features.	8

Map to the Lecture

Chapter	Lecture	Covers
1. No Labels	Introduction; principles (slides 3–11)	vocabulary, examples, principle, individuals/variables table, Fisher's irises
2. How Far Is Near?	Principles (slide 12)	Euclidean, Manhattan, Mahalanobis, χ^2 , Jaccard, similarity, ultrametric
3. Counting Partitions	Principles (slides 13–14)	partitions, combinatorial explosion, hard vs fuzzy
4. Moving Centres	Basic methods (slides 15–19)	k-means algorithm, inertia criterion, convergence, weaknesses
5. The Medoid	Basic methods (slides 20–24)	PAM, Ruspini example, Cassini example
6. The Family Tree	Basic methods (slides 25–28)	divisive vs agglomerative, dendrogram, single / complete / average link, Ward
7. How Many?	Basic methods (slides 29–31)	validation, elbow, partition quality criteria, silhouette
8. Salmon and Sea Bass	Unsupervised vs supervised (slides 32–43)	principle, regression vs discrimination, fish example, evaluation, methodology, parsimony, kNN

A few things in this book go beyond the slides and are flagged as such in the text: k-means++ initialization, the CLARA variant of PAM, fuzzy c-means, the formulas for the Dunn and Davies–Bouldin indices, and the Python examples.

Further reading

- Mirkin, B. (2012). *Clustering: A Data Recovery Approach*. CRC Press.
- Jain, A. K. & Dubes, R. C. (1988). *Algorithms for Clustering Data*. Prentice-Hall.
- Govaert, G. (ed.) (2013). *Data Analysis*. John Wiley & Sons.
- Jain, A. K., Murty, M. N. & Flynn, P. J. (1999). Data clustering: a review. *ACM Computing Surveys*, 31(3), 264–323.
- Jain, A. K. (2010). Data clustering: 50 years beyond K-means. *Pattern Recognition Letters*, 31(8), 651–666.
- Halkidi, M., Batistakis, Y. & Vazirgiannis, M. (2001). On clustering validation techniques. *Journal of Intelligent Information Systems*, 17(2), 107–145.